كلية العلوم والتقنيات سطات
FACULTÉ DES SCIENCES ET TECNHIQUES SETTAT

جامعة الحسن الأول
UNIVERSITÉ HASSAN 1er

Développement Web : *JavaScript*

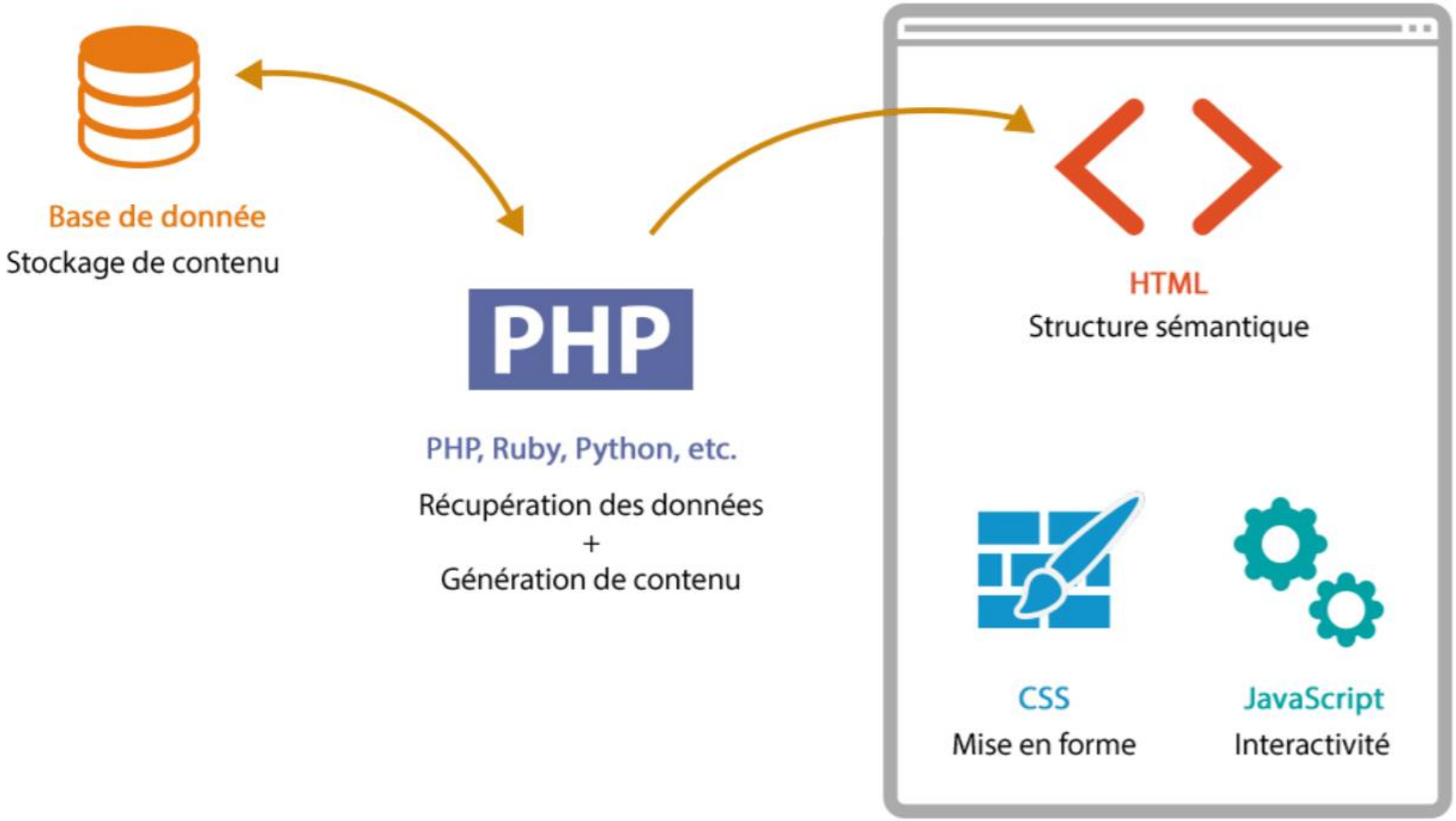
LST: SITD (S5)

Année Universitaire: 2024/2025

Introduction au JavaScript



Page Web : Technologies utilisés pour la création



JavaScript : Définition

– Le **Javascript** est un langage de programmation de script, incorporé aux balises HTML, permet d'améliorer la présentation et l'interactivité des pages Web **côté client**.

– Le Javascript est un langage de programmation **interprété**, c'est-à-dire qu'il a besoin d'un interpréteur pour pouvoir être exécuté.

JavaScript : Définition

– Le javaScript est un **langage dynamique** (HTML et CSS : langages statiques)

⇒ Ce langage va nous permettre de manipuler des contenus HTML ou des styles CSS et de les modifier en fonction de divers *évènements* ou *variables*.

⇒ Un évènement peut être par exemple un clic d'un utilisateur à un certain endroit de la page tandis qu'une variable peut être l'heure de la journée

JavaScript : Exemple

Titre principal

Il est actuellement 11:39:27

Cacher / Afficher l'heure

Clique sur le bouton



Titre principal

Cacher / Afficher l'heure

JavaScript : Utilisation

Généralement, JavaScript sert :

– à interagir avec le document HTML pour contrôler les ressources du client (documents, formulaires, …) DONC moins d'intervention avec le serveur.

– à contrôler les données saisies dans des formulaires HTML avant l'envoi.

– à modifier le contenu des pages web par programmation avec la méthode Ajax (Asynchronous Javascript And XML)

– …

JavaScript : Créer des Scripts

Les scripts Javascript :

- Sont de simples **fichiers ”texte”** (extension **.js**) à créer avec un **éditeur de texte**.
- Sont intégrés au sein des pages web.
- Sont exécutés côté client par le navigateur web.

JavaScript : Créer des Scripts

– La balise **<script>** permet d'indiquer au navigateur le début d'un script. La fin du script sera indiqué par la balise **</script>**.

– Exemple d'insertion d'un code **js** dans une page HTML :

```
< SCRIPT langage="JavaScript"  type="text/javascript" >
        … Instruction;
</SCRIPT>
```

JavaScript : Insertion du code js

1. Interne au document : on peut insérer du code javascript à plusieurs endroits dans un document HTML.
2. Externe au document : le code javascript se trouve dans un fichier séparé portant le plus souvent l'extension **.js** . On devra préciser dans la balise <script> le chemin du fichier:

```
<SCRIPT langage="JavaScript" scr="mon_script.js">
</SCRIPT>
```

=> Elle permet d'inclure le fichier *mon_script.js* puis l'exécuter.

JavaScript : Insertion du code js

1. Interne au document : Le code javascript est généralement inséré entre les balises <head> et </head>

```
<HTML>
   <HEAD>
      <TITLE>Exemple 1</TITLE>
      <SCRIPT TYPE="text/javascript">
      // Du code Javascript
      alert("Hello world !"); // affiche une boîte de dialogue
      </SCRIPT>
   </HEAD>
   <BODY>
   </BODY>
</HTML>
```



JavaScript : Insertion du code js

2. Externe au document :

```
<HTML>
   <HEAD>
      <TITLE>Exemple 3</TITLE>
      <SCRIPT TYPE="text/javascript" SRC="monscript.js"></SCRIPT>
   </HEAD>
   <BODY>
   </BODY>
</HTML>
```

Notions de Base du langage JavaScript



Déclarer une variable

– Avant de pouvoir stocker des informations dans une variable, **il faut la créer**.

⇒ Cette opération s'appelle la **déclaration** de la variable.

– En JavaScript, on déclare une variable à l'aide du mot-clé **let** suivi du nom de la variable.

⇒ Exemple :

```
let a;
```

– Dans les versions précédente du langage, on déclarait une variable à l'aide du mot-clé **var**. C'est toujours possible …

Déclarer une variable

Remarque :

– JavaScript n'impose pas de définir le **type d'une variable**.

– Ce type est déduit de la valeur stockée dans la variable, et peut donc changer au fur et à mesure de l'exécution du programme :

⇒ On dit que JavaScript est un langage à **typage dynamique**.

⇒ D'autres langages comme **C** ou **Java** imposent la définition du type des variables. On parle alors de **typage statique**.

Affecter une valeur à une variable

– Au cours du déroulement du programme, la valeur stockée dans une variable peut changer.

– Pour donner une nouvelle valeur à une variable, on utilise l'opérateur **=** , appelé **opérateur d'affectation.**

```
1  let a;
2  a = 3.14;
```

– On peut également combiner ***déclaration*** et ***affectation*** d'une ***valeur*** en une seule ligne.

```
1  let a = 3.14;
```

Déclarer une Variable Constante

– Si la valeur initiale d'une variable ne changera jamais au cours de l'exécution du programme, cette variable est ce qu'on appelle une constante.

– Il vaut alors mieux la déclarer avec le mot-clé **const** plutôt qu'avec **let**.

```
1  const a = 3.14; // Création d'une variable constante
2  a = 6.28;       // Impossible : a ne peut pas changer de valeur !
```

Portée d'une Variable

On appelle portée (**scope**) **d'une variable** la portion du code source dans laquelle cette variable est visible et donc utilisable.

– **Portée Local** : les variables déclarées au sein d'un bloc ont une portée local (ou bien de type **bloc**) => elles ne sont visibles qu'au sein du bloc de code dans lequel elles sont déclarées.

– **Portée Global** : les variables déclarées hors bloc.

```
1 let num1 = 0;
2 {
3   num1 = 1; // OK : num1 est déclarée dans le bloc parent
4   const num2 = 0;
5 }
6 console.log(num1); // 1
7 console.log(num2); // Erreur : num2 n'est pas visible ici !
```

Notion d'expression

– Une **expression** est un morceau de code qui produit une valeur.

– On crée une expression en combinant des variables, des valeurs et des opérateurs.

– Toute expression *produit une valeur* et *correspond à un certain type*.

```
1 const c = 3; // c est une expression dont la valeur est 3
2 let d = c; // d est une expression dont la valeur est celle de c (3 ici)
3 d = d + 1; // (d + 1) est une expression dont la valeur est celle de d + 1 (4 ici)
4 console.log(d); // 4
```

Notion d'Expression

```
1  let e = 3 + 2 * 4; // e contient 11 (3 + 8)
2  e = (3 + 2) * 4;   // e contient 20 (5 * 4)
```

```
1  const country = "France";
2  console.log(`Je vis en ${country}`); // "Je vis en France"
3
4  const x = 3;
5  const y = 7;
6  console.log(`${x} + ${y} = ${x + y}`); // "3 + 7 = 10"
```

Les Opérateurs

Table 1.2 Opérateurs de comparaison

Egalité	==
Différence	!=
Inférieur, supérieur	< ; >
Inférieur ou égal, supérieur ou égal	<= ; >=

Table 1.1 Opérateurs arithmétiques

Opération	Signe
Addition	+
Soustraction	-
Multiplication	*
Division	/
Modulo	%
Negation	-
Incrément	++
Décrément	--

Table 1.3 Opérateurs logiques

ET	&&
OU	| |
NON	!

Les Interactions avec l'utilisateur

– **prompt()** : Permet de poser une question ouverte à l'utilisateur. Elle ouvre une boite de dialogue avec une zone saisie et 2 boutons OK et Annuler, retourne l'information lue.

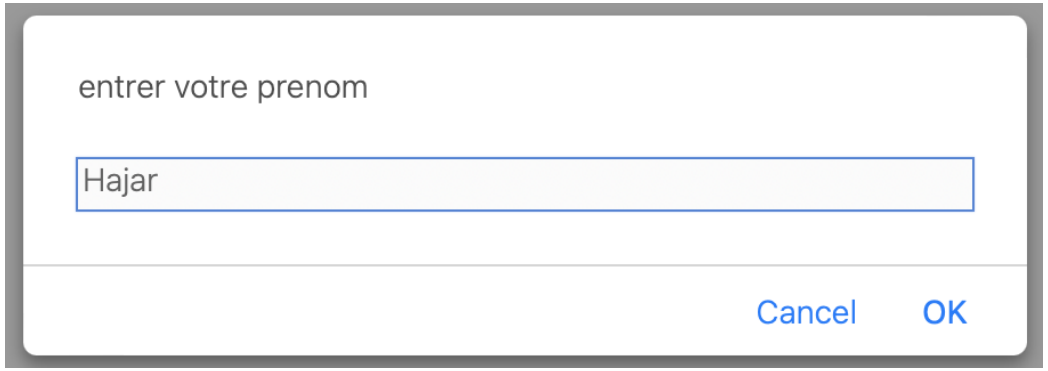
– **alert()** : Permet d'écrire un message à l'utilisateur dans une fenêtre.

– **document.write()** : Afficher un message dans le navigateur.

– **confirm()** : Permet de poser une question à l'utilisateur. Il ouvre une boite de dialogue avec 2 boutons OK et Annuler, retourne un booléen

Les Interactions avec l'utilisateur

```
<HTML>
<HEAD>
<TITLE> Utilisation de prompt() et d'alert() </TITLE>
</HEAD>
<BODY>
    <SCRIPT LANGAGE="Javascript">
        const prenom = prompt("Entrez votre prénom :");
        alert(`Bonjour, ${prenom}`);
    </SCRIPT>
</BODY>
</HTML>
```





Les Interactions avec l'utilisateur

```
<HTML>
<HEAD>
<TITLE> Utilisation de write  </TITLE>
</HEAD>
<BODY>
    <SCRIPT type="text/javascript" LANGAGE="Javascript">
        alert("chargement du nouveau contenu");
        document.write("<h1>Assez de l'ancien contenu, passons au nouveau !</h1>");
    </SCRIPT>
</BODY>
</HTML>
```



Assez de l'ancien contenu, passons au nouveau !

Structures Conditionnelles et Répétitives



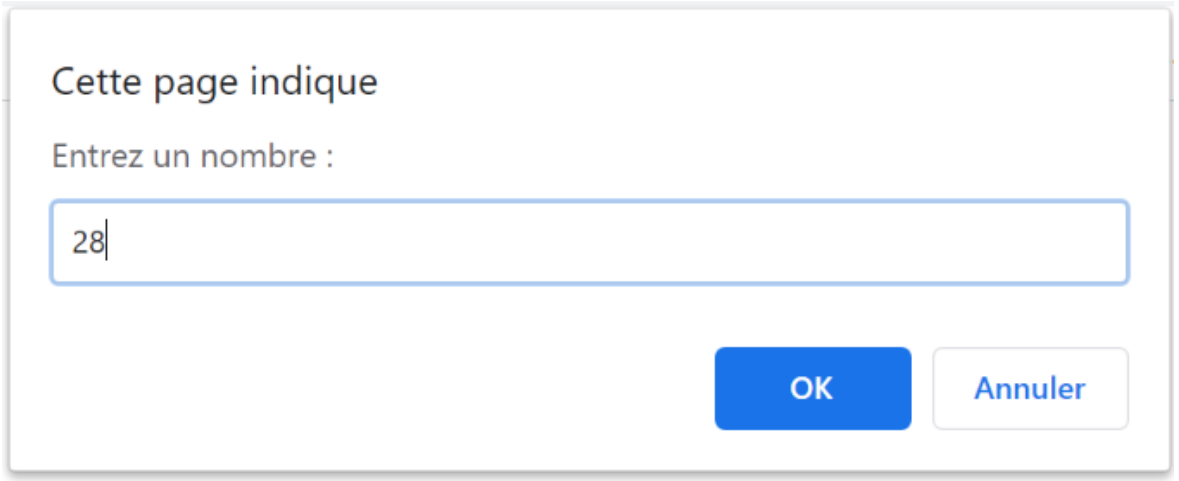
Les Structures Conditionnelles : if … else

```
if (condition)
{   liste d'instructions; }
else
{   liste d'instructions; }
```

La "condition" doit être une expression booléenne évaluable. Si elle est VRAI, alors la première série d'instructions est exécutée, si elle est FAUSSE, alors c'est la seconde série qui est exécutée.

Les Structures Conditionnelles : if … else

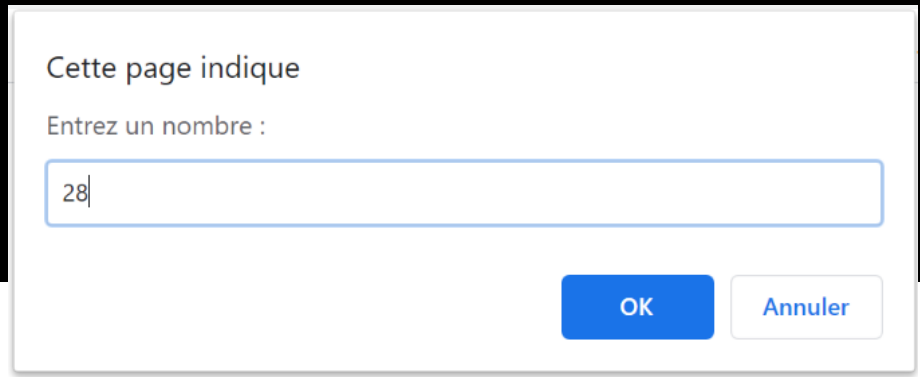
Exercice: Ecrire un code Javascript qui teste si un nombre (saisi par l'utilisateur) est positif, négatif ou nul.



28 est positif

Les Structures Conditionnelles : if … else

```
<HTML>
<HEAD>
<TITLE> Utilisation de structure conditionnelle  </TITLE>
    <script LANGAGE="javascript" type="text/javascript" >
        const nombre = Number(prompt("Entrez un nombre :"));
        if (nombre > 0) {
          document.write("<h2>" + nombre + " est positif </h2>");
        } else if (nombre < 0) {
          document.write("<h2>" + nombre + " est négatif </h2>");
        } else {
          document.write("<h2>" + nombre + " est nul </h2>");
        }
    </script>
</HEAD>
<BODY>
</BODY>
</HTML>
```



28 est positif

Les Structures Conditionnelles : Switch

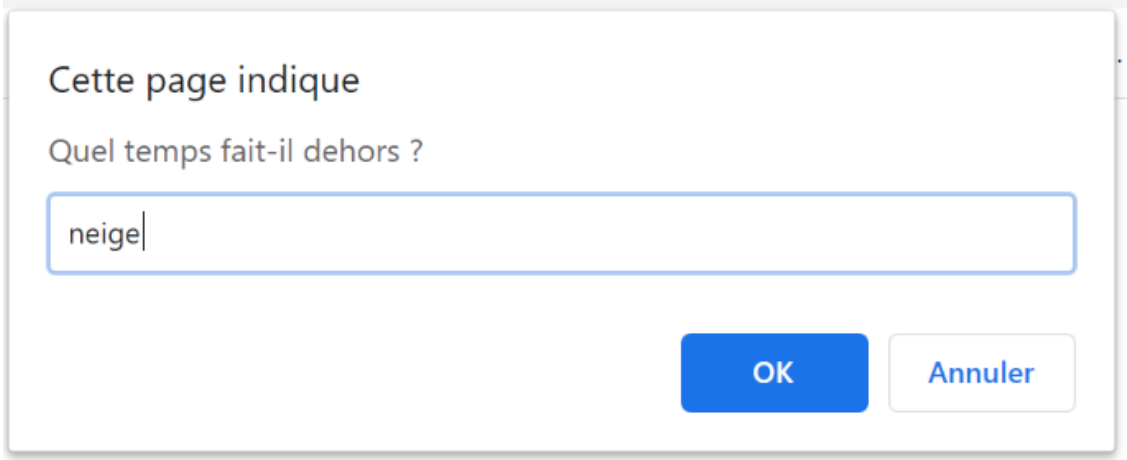
```
switch (expression) {
  case valeur1:
    // instructions exécutées quand expression vaut valeur1
    break;
  case valeur2:
    // instructions exécutées quand expression vaut valeur2
    break;
  ...
  default:
    // instructions exécutées quand aucune des valeurs ne correspond
}
```

Remarque : Les instructions **break;** dans les blocs **case** sont indispensables pour sortir du **switch** et éviter de passer d'un bloc à un autre.

Les Structures Conditionnelles : Switch

Exercice:

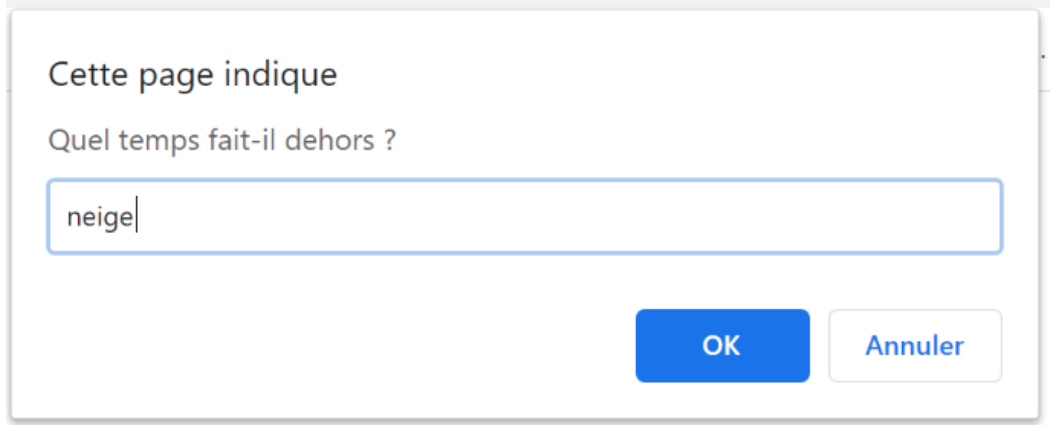
Ecrire un code Javascript qui affiche un conseil à l'utilisateur selon la nature de la météo (*Soleil*, *Vent Neige ou Pluie*).



Restez au chaud à la maison.

Les Structures Conditionnelles : Switch

```
<HTML>
<HEAD>
<TITLE> Utilisation de structure conditionnelle  </TITLE>
    <script LANGAGE="javascript" type="text/javascript" >
        const meteo = prompt("Quel temps fait-il dehors ?");
        switch (meteo) {
          case "soleil":
            document.write("Sortez en t-shirt.");
            break;
          case "vent":
            document.write("Sortez en pull.");
            break;
          case "pluie":
            document.write("Sortez en blouson.");
            break;
          case "neige":
            document.write("Restez au chaud à la maison.");
            break;
          default:
            document.write("Je n'ai pas compris !");
        }
    </script>
</HEAD>
<BODY>
</BODY>
</HTML>
```



Restez au chaud à la maison.

Les Structures Répétitives : While

```
1 while (condition) {
2   // instructions exécutées tant que la condition est vérifiée
3 }
```

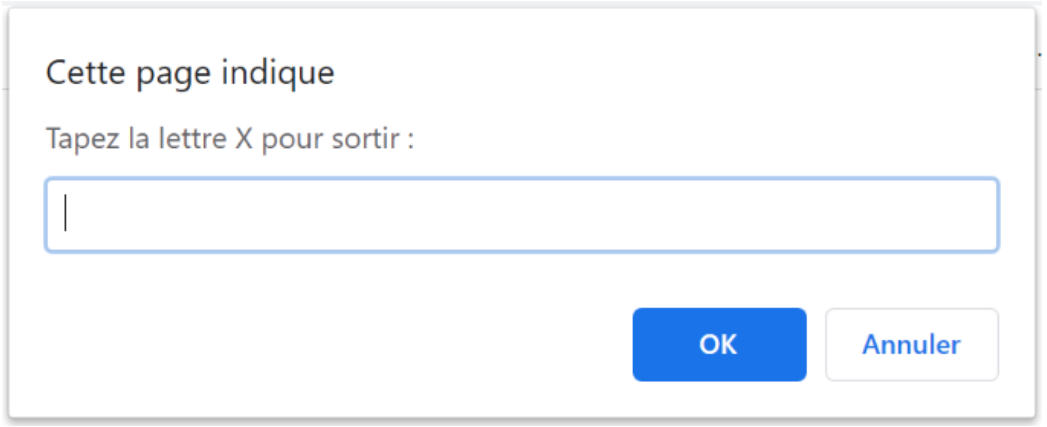
– La boucle while permet de répéter des instructions **tant qu'une condition est vérifiée**.

- Si elle est vraie, les instructions du bloc de code associé au while sont exécutées. Ensuite, l'exécution revient au niveau du while et la condition est à nouveau vérifiée.
- Si elle est fausse, les instructions du bloc ne sont pas exécutées et le programme continue juste après le bloc while.

Les Structures Répétitives : While

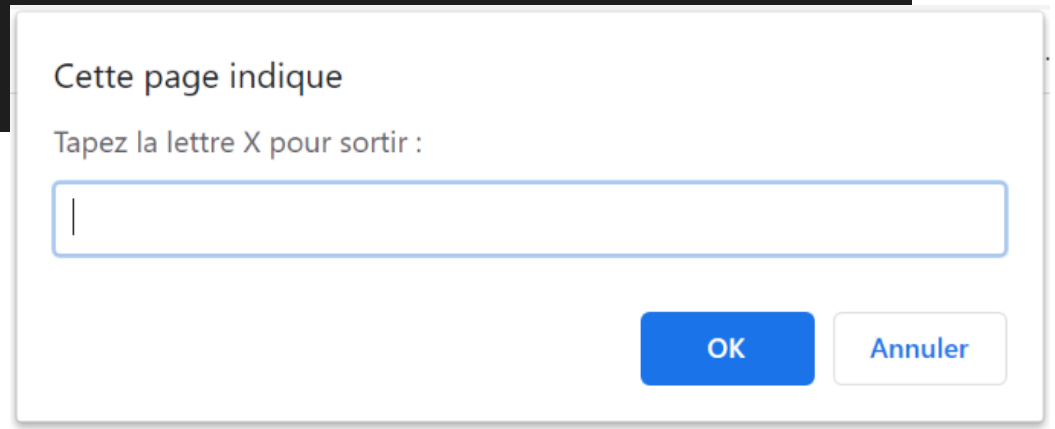
Exercice :

Ecrire un code Javascript qui permet de demander à l'utilisateur de saisir une lettre. Si la lettre est égale à X, le programme s'arrête.



Les Structures Répétitives : While

```
1   <html>
2   <HEAD>
3   <TITLE> Utilisation de boucle while </TITLE>
4   <script LANGAGE="javascript" type="text/javascript">
5   let lettre= '';
6   while (lettre != "X") {
7   lettre = prompt ("Tapez la lettre X pour sortir : ");}
8   </script>
9   </HEAD>
10  <BODY>
11  </BODY>
12  </html>
```



Les Structures Répétitives : For

```
<HTML>
<HEAD>
<TITLE> Utilisation de boucle for </TITLE>
    <script LANGAGE="javascript" type="text/javascript" >
        for (let compteur = 1; compteur <= 5; compteur++) {
            document.write("<h4>" + compteur + "</h4>");
        }
    </script>
</HEAD>
<BODY>
</BODY>
</HTML>
```

1

2

3

4

5

Structurer le code avec les Fonctions



Déclaration d'une Fonction

– La **déclaration d'une fonction** s'effectue à l'aide du mot-clé function suivi du nom de la fonction et d'une paire de parenthèses contenant les paramètres de la fonctions (s'ils existent).

– Les instructions qui composent la fonction constituent le **corps** de la fonction.

```
function   nomfonction ( param1, ..., paramN)
{          // code JavaScript …
           return variable_ou_valeur ;
}
```

Appel d'une Fonction

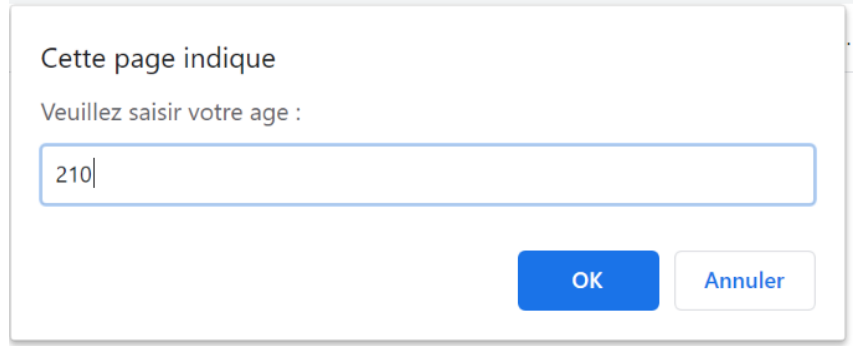
L'appel d'une fonction déclenche l'exécution des instructions qui la constituent, puis l'exécution reprend à l'endroit où la fonction a été appelée.

⇒ Syntaxe :

let res = **nomFonction**(*var1, val2, varN*) ;

Fonction : Exemple

```
<HTML>
<HEAD>
<TITLE> Utilisation de boucle for </TITLE>
    <script LANGAGE="javascript" type="text/javascript" >
        //Déclaration de la fonction
        function isHumanAge(age){
            if ((age < 0) || (age > 120)) { return false ; }
            else { return true ; }
        }
        //utiliser la fonction déclarée
        let age = prompt("Veuillez saisir votre age :");
        if(!isHumanAge(age))
        {
           alert("Vous ne pouvez pas avoir " + age + " ans !");
        }
    </script>
</HEAD>
<BODY>
</BODY>
</HTML>
```





Fonction : Arguments variables

```
<HTML>
<HEAD>
<TITLE> Exemple JavaScript </TITLE>
    <script LANGAGE="javascript" type="text/javascript" >
        function somme()
        {
            var arg = somme.arguments;
            var argL = somme.arguments.length;
            var result=0;
            for(var i=0 ; i < argL ; i++)
            { result += arg[i]; }
            return result;
        }
        //Appel de la fonction
        let R1 = somme(1,2,3);
        let R2 = somme(8);
        document.write("l'appel de somme(1,2,3) retourne : " + R1 + "<br>");
        document.write("l'appel de somme(8) retourne : " + R2 + "<br>");
    </script>
</HEAD>
<BODY>
</BODY>
</HTML>
```

l'appel de somme(1,2,3) retourne : 6
l'appel de somme(8) retourne : 8

Fonction Prédéfinies

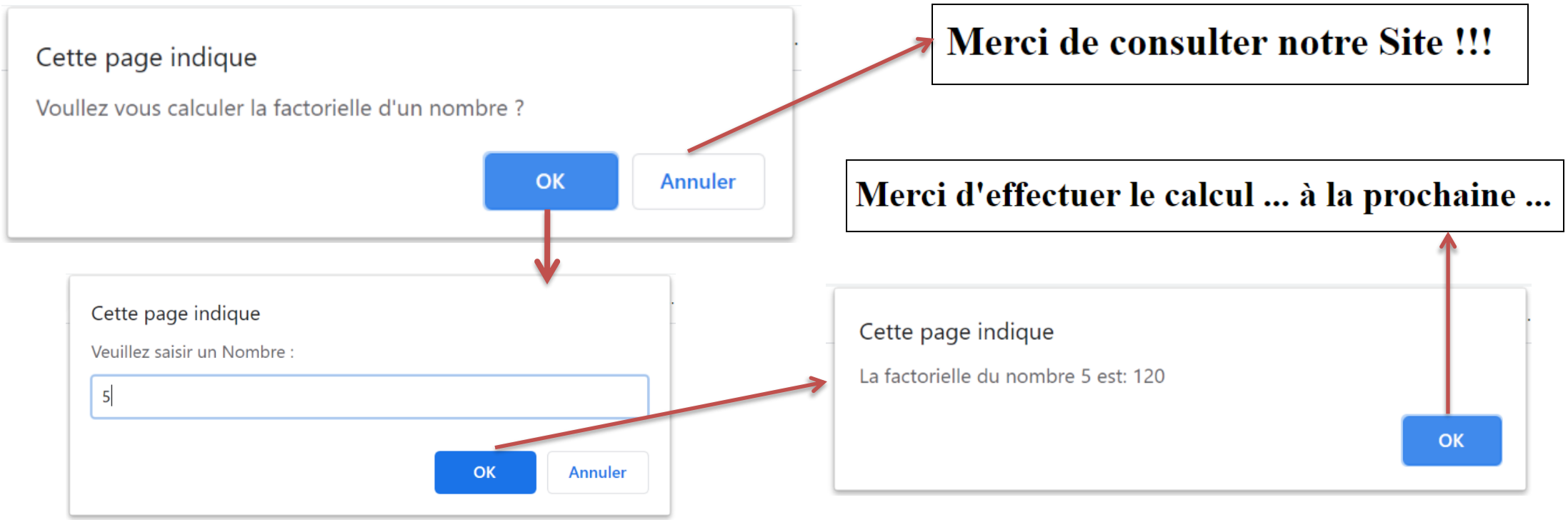
– Le langage vous propose un nombre important de fonctions qui répondent à des besoins variés.

– Exemple de FP pour la manipulation des chaines de caractères :

- **charAt(i)** retourne le ième caractère de la chaîne.
- **toLowerCase()** convertit la chaîne en minuscules.
- **toUpperCase()** convertit la chaîne en majuscules.
- **replace(exp, ch)** remplace les occurrences de exp par ch.
- …

Exercice d'application : 1

Ecrire un script qui permet d'afficher le **factoriel** d'un nombre saisi par l'utilisateur après avoir lui demander la confirmation d'action du calcul.



Exercice d'application : 1

```
<script LANGAGE="javascript" type="text/javascript">

    function fact(nbr){
        var F=1,j=1;
        while(j<=nbr){
            F*=j;
            j++;
        }
        return F;
    }
    if(confirm("Voullez vous calculer la factorielle d'un nombre ?")){
        let N = Number(prompt("Veuillez saisir un Nombre :"));
        let F = fact(N);
        alert("La factorielle du nombre "+ N +" est: "+ F);
        document.write("<h2> Merci d'effectuer le calcul ... à la prochaine ...</h2>");
    }else {
        document.write("<h2> Merci de consulter notre Site !!!</h2>");
    }

</script>
```

Programmation Evénementielle



Programmation événementielle : Définition

– La **programmation événementielle** permet la gestion d'événements.

– Un **événement** est généralement associé à une action de l'utilisateur.

⇒ appui sur une touche, click ou déplacement de la souris, ...

– En **HTML classique**, il y a très peu d'événements qui sont gérés : click sur un lien ou sur un bouton de formulaire. Leur gestion est automatique et non modifiable.

– Le **javascript** va permettre de gérer et contrôler d'autres événements.

Programmation événementielle : Principe

– En javascript, on va pouvoir associer à chaque événement une action : une fonction, cad le gestionnaire d'événement (EVENT HANDLER).

Exemple :

L'objet (Object)	Événement (Event)	Gestionnaire d'événement (Event Handler)
Bouton	Clic de souris	Envoi du formulaire

Les événements en JavaScript

– **Click** : L'utilisateur clique sur un bouton, un lien ou tout autre élément.

– **Load** : La page est chargée par le navigateur.

– **Unload** : L'utilisateur quitte la page

– **MouseOver** : L'utilisateur place le pointeur de la souris sur un élément.

– **MouseOut** : Le pointeur de la souris quitte un lien ou tout autre élément.

Les événements en JavaScript

– **Focus** : Un élément de formulaire a le focus (devient la zone d'entrée active).

– **Blur** : Un élément de formulaire perd le focus

– **Change** : La valeur d'un champ de formulaire est modifiée.

– **Select** : L'utilisateur sélectionne un champ dans un élément de formulaire.

– **Submit** : L'utilisateur clique sur le bouton submit pour envoyer un formulaire.

– **Keydown** : L'utilisateur appuie sur une touche.

Gestionnaire d'événement

– La syntaxe pour installer le gestionnaire d'événement :

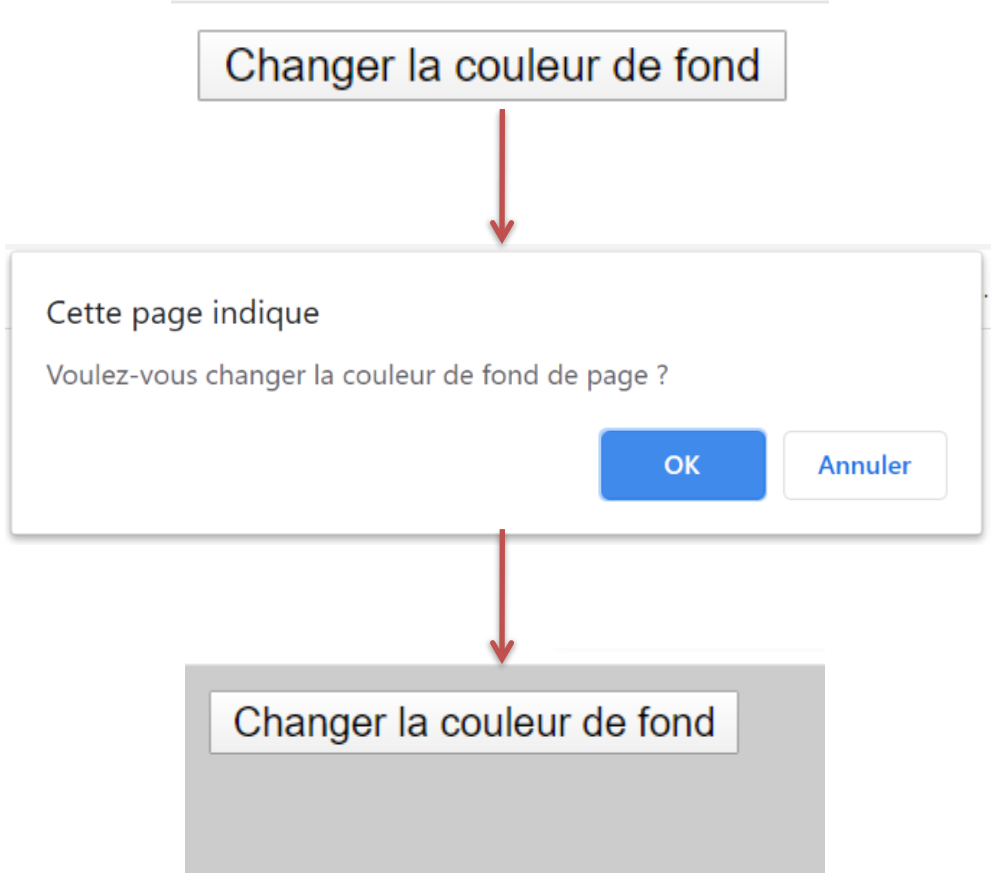
onEvenement = fonction()

```
<HTML>
<HEAD>
<TITLE> Exemple JavaScript </TITLE>
</HEAD>
<BODY>
        <FORM>
            <INPUT TYPE="button" VALUE="Cliquez ici SVP"
            onClick="alert('Vous avez bien clique sur le bouton !!! :D')">
        </FORM>
</BODY>
</HTML>
```



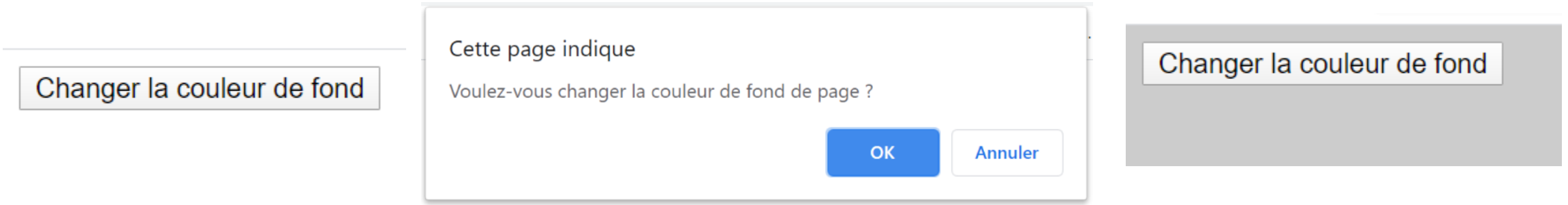
Gestionnaire d'événement

Exercice d'application :



Gestionnaire d'événement

```
<BODY>
    <form>
        <input type="button" value="Changer la couleur de fond" onClick="ConfirmMessage()">
    </form>
    <script type="text/javascript">
        function ConfirmMessage() {
            if (confirm("Voulez-vous changer la couleur de fond de page ?")) {
                // Clic sur OK
                document.body.style.backgroundColor="#ccc";
            }
        }
    </script>
</BODY>
```



Événements et Objets

Liste des objets auxquels correspondent des gestionnaires d'événement :

- Fenêtre : onLoad, onUnload
- Lien hypertexte : onClick, onMouseOver, onMouseOut
- Élément de texte : onBlur, onChange, onFocus, onSelect
- Élément de zone de texte : onBlur,onChange,onFocus,onSelect
- Éléments bouton, case à cocher, bouton radio : onClick
- Liste de sélection : onBlur, onChange, onFocus
- Bouton Submit ou Reset : onClick

Gestion des Formulaires avec JavaScript



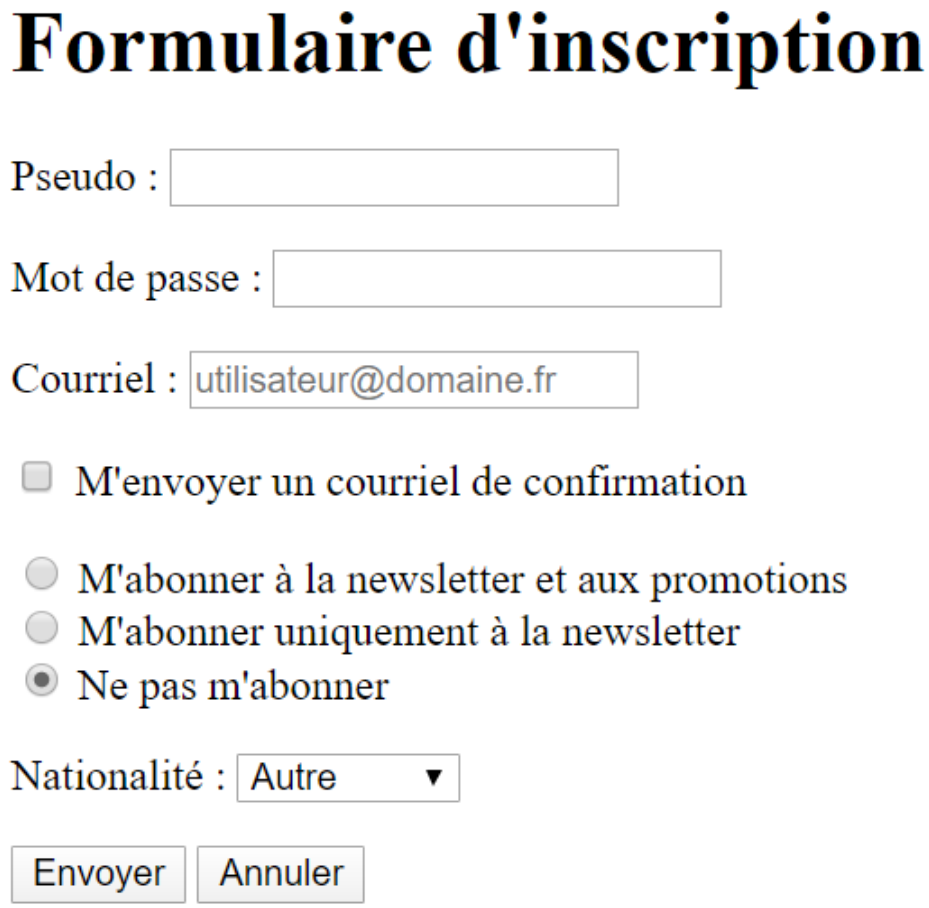
Gestion des Formulaires avec JavaScript

– Grâce à JavaScript, nous allons pouvoir **manipuler le formulaire et ses données** directement **côté navigateur client**, avant (éventuellement) d'envoyer ces données vers un serveur externe.

– Ainsi, on pourra avertir immédiatement l'utilisateur en cas de saisie erronée, ou bien lui proposer une liste de suggestions au fur et à mesure de sa frappe, et bien d'autres choses encore.

Gestion des Formulaires avec JavaScript

Nous allons partir du formulaire conçu dans le cours html :



Gestion des Formulaires : Les zones de texte

➢ Accès à la valeur saisie :

En JavaScript, on accède à la valeur d'une zone de texte en utilisant:

document.getElementById.value

```
1 <label for="pseudo">Pseudo</label> :
2 <input type="text" name="pseudo" id="pseudo" required>
```

Formulaire d'inscription

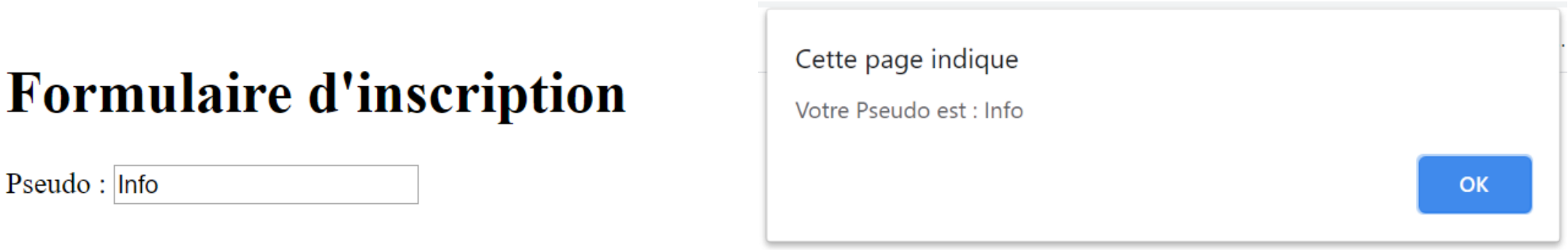
Pseudo :

Gestion des Formulaires : Les zones de texte

- **Accès à la valeur saisie :** Exemple

```
<form action="javascript:gestionnaire()">
```

```
<script LANGAGE="javascript" type="text/javascript">
    let pseudoElt = document.getElementById("pseudo");
    function gestionnaire() {
        alert("Votre Pseudo est : " + pseudoElt.value);
    }
</script>
```



Gestion des Formulaires : Les zones de texte

➢ Gestion du Focus :

– Lorsqu'une **zone de texte** est la cible de saisie, on dit que cette zone possède le **focus**.

– L'utilisateur d'un formulaire qui clique sur une zone de saisie pour y taper une information déclenche l'apparition d'un événement de type onFocus.

– Inversement, le changement de cible de saisie provoque l'apparition d'un événement de type onBlur sur l'ancienne zone qui avait le focus.

Gestion des Formulaires : Eléments d'options

– Ces éléments permettent à l'utilisateur de faire un choix parmi plusieurs possibilités (cases à cocher, bouton radio ou liste déroulante).

– Ils ont en commun de déclencher un événement de type **Change** lorsque l'utilisateur modifie son choix.

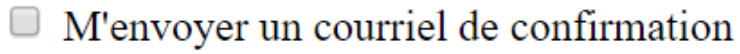
Gestion des Formulaires : Eléments d'options

➢ Case à cocher :

– Lorsque la case change de valeur, l'objet Event associé à l'événement dispose d'une propriété booléenne checked qui indique le nouvel état de la case (cochée/décochée).

– L'exemple de code ci-dessous gère les événements change sur la case à cocher pour afficher dans la console le choix effectué lorsque l'utilisateur clique dans la case ou sur le libellé associé.

```
1 <input type="checkbox" name="confirmation" id="confirmation">
2 <label for="confirmation">M'envoyer un courriel de confirmation</label>
```

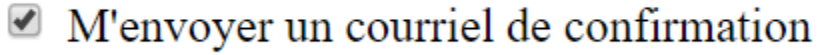


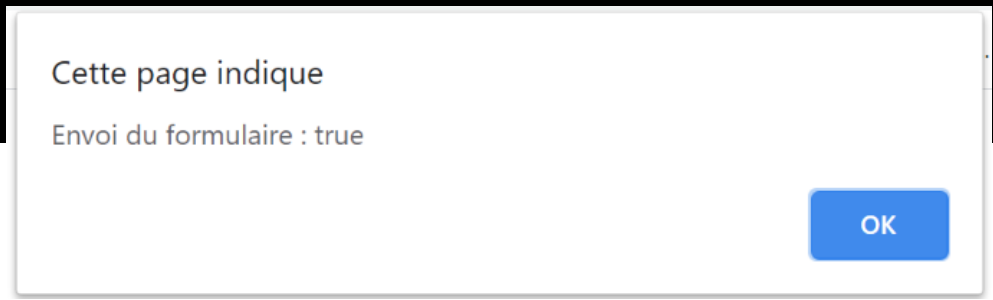
Gestion des Formulaires : Eléments d'options

➢ **Case à cocher** *(Exemple)* **:** gère l'événement **change** sur la case à cocher pour afficher le choix effectué lorsque l'utilisateur clique dans la case.

```
<input type="checkbox" name="confirmation" id="confirmation" onchange="confir()">
<label for="confirmation" >M'envoyer un courriel de confirmation</label>

<script LANGAGE="javascript" type="text/javascript">
function confir(){
    let rep = document.getElementById("confirmation").checked;
    alert("Envoi du formulaire : " + rep);
}
</script>
```





Gestion des Formulaires : Eléments d'options

➢ Bouton radio :

```
1 <input type="radio" name="abonnement" id="abonewspromo" value="abonewspromo">
2 <label for="abonewspromo">M'abonner à la newsletter et aux promotions</label>
3 <br>
4 <input type="radio" name="abonnement" id="abonews" value="abonews">
5 <label for="abonews">M'abonner uniquement à la newsletter</label>
6 <br>
7 <input type="radio" name="abonnement" id="aborien" value="aborien" checked>
8 <label for="aborien">Ne pas m'abonner</label>
9 <br>
```

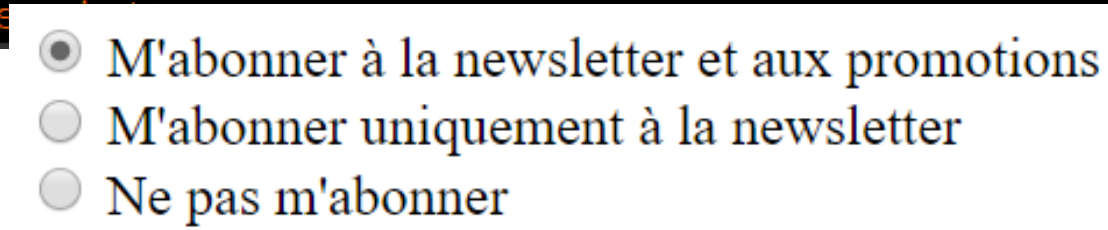
- ○ M'abonner à la newsletter et aux promotions
- ○ M'abonner uniquement à la newsletter
- ◉ Ne pas m'abonner

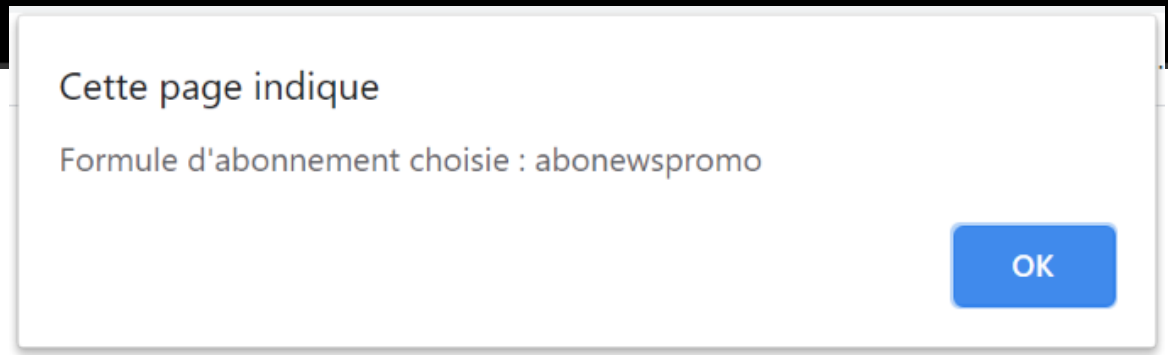
Gestion des Formulaires : Eléments d'options

- **Bouton radio** : Ajout d'evnt javaScript

```
<input type="radio" name="abonnement" id="abonewspromo" value="abonewspromo" onchange="gestionRadio()">
```

```
<script LANGAGE="javascript" type="text/javascript">
function gestionRadio(){
    let aboElts = document.getElementsByName("abonnement");
    for (let i = 0; i < aboElts.length; i++) {
        if(aboElts[i].checked){alert("Formule d'abonnement choisie : " + aboElts[i].value);}
    }
}
</s
```

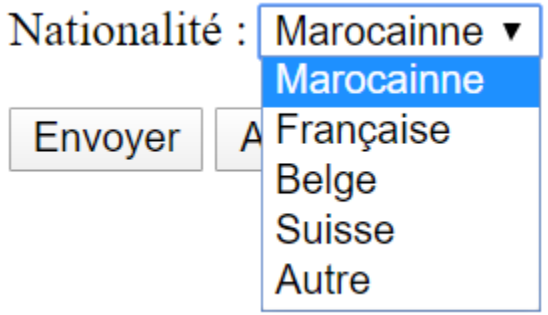




Gestion des Formulaires : Eléments d'options

➢ Listes déroulantes :

```
<label for="nationalite">Nationalité :</label>
<select name="nationalite" id="nationalite">
    <option value="MR" selected>Marocainne</option>
    <option value="FR" >Française</option>
    <option value="BE">Belge</option>
    <option value="SUI">Suisse</option>
    <option value="XX">Autre</option>
</select>
```



Gestion des Formulaires : Eléments d'options

➢ Listes déroulantes : *Exemple*

```
<label for="nationalite" >Nationalité :</label>
<select name="nationalite" id="nationalite" onchange="nation()">
    <option value="MR" selected>Marocainne</option>
    <option value="FR" >Française</option>
    <option value="BE">Belge</option>
    <option value="SUI">Suisse</option>
    <option value="XX">Autre</option>
</select>

<script LANGAGE="javascript" type="text/javascript">
    let nat = document.getElementById("nationalite");
    function nation() {
        alert("Votre nationnalité est : " + nat.value);
    }
</script>
```

